부록 · LangChain4j · Spring AI

# Spring Boot Java + Kotlin 다음 단계

**업비트 실제 XRP 시세 REST API 연동**

## 1. 이번 단계의 목표

이전 단계에서는 XRP 가격을 코드에 직접 입력했습니다. 이번에는 업비트 공개 REST API에서 KRW-XRP 실제 현재가를 가져와 Spring Boot REST API로 출력합니다.

```text
업비트 API
   ↓
실제 XRP 현재가
   ↓
Spring Boot
   ↓
UpbitService
   ↓
XrpController
   ↓
http://localhost:8080/api/xrp
```

이번 단계에서는 JPA, MariaDB, Repository, LangChain4j, Gemma3를 넣지 않습니다. DTO → Service → Controller 흐름부터 확실히 익힙니다.

## 2. 프로젝트 구조

```text
upbit-ai/
├─ controller/
│  └─ XrpController
├─ service/
│  └─ UpbitService
├─ dto/
│  └─ UpbitTickerDto
└─ UpbitAiApplication
```

핵심 구조: Controller → Service → 외부 Upbit API

## 3. Java Spring Boot 버전

### 3-1. UpbitTickerDto.java

```java
package com.wiz.upbit.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public record UpbitTickerDto(
        String market,

        @JsonProperty("trade_price")
        Double tradePrice
) {
}
```

DTO는 업비트가 보내는 JSON 데이터를 Java 객체로 받아주는 '그릇'입니다. trade_price를 Java의 tradePrice에 연결하기 위해 @JsonProperty를 사용합니다.

### 3-2. UpbitService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.UpbitTickerDto;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
                .baseUrl("https://api.upbit.com")
                .build();
    }

    public UpbitTickerDto getXrpPrice() {

        UpbitTickerDto[] result = restClient.get()
                .uri("/v1/ticker?markets=KRW-XRP")
                .retrieve()
                .body(UpbitTickerDto[].class);

        return result[0];
    }
}
```

restClient.get()은 업비트 서버에 GET 요청을 보내는 부분입니다. /v1/ticker?markets=KRW-XRP는 원화 기준 XRP 현재가 정보를 요청합니다.

### 3-3. XrpController.java

```java
package com.wiz.upbit.controller;

import com.wiz.upbit.dto.UpbitTickerDto;
import com.wiz.upbit.service.UpbitService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final UpbitService upbitService;

    public XrpController(UpbitService upbitService) {
        this.upbitService = upbitService;
    }

    @GetMapping
    public UpbitTickerDto getPrice() {
        return upbitService.getXrpPrice();
    }
}
```

## 4. Kotlin Spring Boot 버전

### 4-1. UpbitTickerDto.kt

```kotlin
package com.wiz.upbit.dto

import com.fasterxml.jackson.annotation.JsonProperty

data class UpbitTickerDto(
    val market: String,

    @JsonProperty("trade_price")
    val tradePrice: Double
)
```

### 4-2. UpbitService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.UpbitTickerDto
import org.springframework.stereotype.Service
import org.springframework.web.client.RestClient

@Service
class UpbitService(
    builder: RestClient.Builder
) {

    private val restClient = builder
        .baseUrl("https://api.upbit.com")
        .build()

    fun getXrpPrice(): UpbitTickerDto {

        val result = restClient
            .get()
            .uri("/v1/ticker?markets=KRW-XRP")
            .retrieve()
            .body(Array<UpbitTickerDto>::class.java)

        return result!![0]
    }
}
```

### 4-3. XrpController.kt

```kotlin
package com.wiz.upbit.controller

import com.wiz.upbit.dto.UpbitTickerDto
import com.wiz.upbit.service.UpbitService
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RestController

@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val upbitService: UpbitService
) {

    @GetMapping
    fun getPrice(): UpbitTickerDto {
        return upbitService.getXrpPrice()
    }
}
```

## 5. Java ↔ Kotlin 비교표

| 항목 | Java | Kotlin |
| --- | --- | --- |
| DTO | record | data class |

| 항목 | Java | Kotlin |
|---|---|---|
| 변수 | Double tradePrice | val tradePrice: Double |
| 함수 | public UpbitTickerDto getXrpPrice() | fun getXrpPrice(): UpbitTickerDto |
| 생성자 주입 | 생성자를 별도로 작성 | 클래스 선언부에서 바로 작성 |
| GET 호출 | restClient.get() | restClient.get() |
| @Service | 동일 | 동일 |
| @RestController | 동일 | 동일 |
| 특징 | 명시적이고 코드가 조금 김 | 간결하고 코드량이 적음 |

## 6. 실행 및 확인

Spring Boot를 실행한 뒤 브라우저 또는 Postman에서 다음 주소를 호출합니다.

```text
http://localhost:8080/api/xrp
```

```text
예상 JSON 형태:
{
  "market": "KRW-XRP",
  "tradePrice": 4200.0
}
```

실제 tradePrice 값은 호출 시점의 업비트 XRP 시세에 따라 달라집니다.

## 7. 꼭 이해해야 할 흐름

```text
브라우저
   ↓
XrpController
   ↓
UpbitService
   ↓
Upbit REST API
   ↓
JSON
   ↓
UpbitTickerDto
   ↓
Controller
   ↓
브라우저에 JSON 출력
```

## 8. 다음 학습 단계

다음에는 현재가 1개가 아니라 XRP 최근 캔들 데이터를 가져와 MA5, MA20, RSI, 거래량을 계산합니다. 그 다음 계산 결과를 LangChain4j → Ollama → Gemma3로 전달하면 AI 시세 분석 구조가 완성됩니다.

```text
업비트 XRP 캔들
   ↓
최근 데이터
   ↓
MA5 / MA20 / RSI / 거래량
   ↓
LangChain4j
   ↓
Ollama
http://localhost:11434
   ↓
Gemma3
   ↓
상승 / 하락 / 횡보 분석
```

---

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · LangChain4j · Spring AI

# LangChain4j + Spring Boot Java · Kotlin 비교 학습자료

**업비트 XRP 시세 분석 · Ollama Gemma3 연동 예제**

Ollama 서버:

```
http://localhost:11434
```
text

## 1. 전체 구조

업비트 API → XRP(KRW-XRP) 시세/캔들

→ Spring Boot → LangChain4j

→ Ollama (http://localhost:11434) → Gemma3

→ 상승 / 하락 / 횡보 분석 → REST API 결과

처음에는 DB, JPA, Repository를 넣지 않고 '업비트 데이터 → AI 분석 → REST API 출력' 흐름부터 성공시키는 것이 가장 이해하기 쉽습니다.

## 2. Spring Boot Java + Kotlin 비교표

| 항목 | Spring Boot Java | Spring Boot Kotlin |
|---|---|---|
| 코드 길이 | 비교적 김 | 짧고 간결함 |
| LangChain4j | 매우 좋음 | 매우 좋음 |
| Ollama / Gemma3 | 연동 가능 | 연동 가능 |
| 업비트 REST API | 연동 가능 | 연동 가능 |
| DTO | class / record | data class |
| Controller | 조금 길어짐 | 간결함 |
| JPA | 자료가 매우 많음 | 사용 편리 |
| 학습 자료 | 매우 많음 | 많음 |
| 추천 용도 | 기본 원리 학습 | 주력 개발에 추천 |

## 3. 핵심 구성요소 3개

① UpbitService — 업비트에서 XRP 시세와 캔들 데이터를 가져옴

② XrpAiService — LangChain4j를 통해 Gemma3에게 분석을 요청함

③ XrpController — 브라우저나 앱에서 호출할 REST API를 제공함

## 4. 공통 application.properties

```properties
server.port=8080

ollama.base-url=http://localhost:11434
ollama.model=gemma3
```

base-url은 Ollama가 실행 중인 서버 주소이고, model은 Ollama에 설치된 정확한 모델명을 지정합니다. 실제 설치 이름이 gemma3:4b 등이라면 그 이름으로 변경합니다.

## 5. Java 예제

### 5-1. OllamaConfig.java

```java
@Configuration
public class OllamaConfig {

    @Bean
    public ChatModel chatModel(
            @Value("${ollama.base-url}") String baseUrl,
            @Value("${ollama.model}") String modelName) {

        return OllamaChatModel.builder()
                .baseUrl(baseUrl)
                .modelName(modelName)
                .temperature(0.2)
                .build();
    }
}
```

### 5-2. XrpAiService.java

```java
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String predict(String priceData) {

        String prompt = """
                당신은 암호화폐 시세 분석 AI입니다.

                다음은 업비트 XRP 가격 데이터입니다.

                %s

                현재 추세:
                예상 방향: 상승/하락/횡보
                이유:
                주의사항:
                """.formatted(priceData);

        return chatModel.chat(prompt);
    }
}
```

### 5-3. XrpController.java

```java
@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final XrpAiService aiService;

    public XrpController(XrpAiService aiService) {
        this.aiService = aiService;
    }

    @GetMapping("/predict")
    public String predict() {

        String data = """
                현재가: 4,200원
                5일 이동평균: 4,150원
                20일 이동평균: 4,000원
                거래량: 전일 대비 35% 증가
                RSI: 62
                """;

        return aiService.predict(data);
    }
}
```

## 6. Kotlin 예제

### 6-1. OllamaConfig.kt

```kotlin
@Configuration
class OllamaConfig {

    @Bean
    fun chatModel(
        @Value("\${ollama.base-url}") baseUrl: String,
        @Value("\${ollama.model}") modelName: String
    ): ChatModel {

        return OllamaChatModel.builder()
            .baseUrl(baseUrl)
            .modelName(modelName)
            .temperature(0.2)
            .build()
    }
}
```

### 6-2. XrpAiService.kt

```kotlin
@Service
class XrpAiService(
    private val chatModel: ChatModel
) {

    fun predict(priceData: String): String {

        val prompt = """
            당신은 암호화폐 시세 분석 AI입니다.

            다음은 업비트 XRP 가격 데이터입니다.

            $priceData

            현재 추세:
            예상 방향: 상승/하락/횡보
            이유:
            주의사항:
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

### 6-3. XrpController.kt

```kotlin
@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val aiService: XrpAiService
) {

    @GetMapping("/predict")
    fun predict(): String {

        val data = """
            현재가: 4,200원
            5일 이동평균: 4,150원
            20일 이동평균: 4,000원
            거래량: 전일 대비 35% 증가
            RSI: 62
        """.trimIndent()

        return aiService.predict(data)
    }
}
```

## 7. Java ↔ Kotlin 핵심 차이

| Java | Kotlin |
|---|---|
| private final ChatModel chatModel; | private val chatModel: ChatModel |
| public String predict(String data) | fun predict(data: String): String |
| 생성자를 별도로 작성 | 주 생성자에서 바로 의존성 주입 |
| 코드가 명시적이고 길어짐 | 같은 기능을 더 짧게 작성 |

## 8. 실제 업비트 XRP 데이터로 발전시키기

위 예제는 학습을 위해 가격 데이터를 직접 넣었습니다. 실제 프로젝트에서는 UpbitService가 KRW-XRP 캔들을 가져오고 IndicatorService가 MA5, MA20, RSI, 거래량 등을 계산한 뒤 그 결과를 Gemma3에 전달합니다.

Upbit REST API → KRW-XRP 최근 캔들 수집

IndicatorService → MA5 / MA20 / RSI / 거래량 계산

XrpAiService → 계산된 지표를 Gemma3에 전달

Gemma3 → 상승 / 하락 / 횡보와 이유 분석

XrpController → JSON 형태로 결과 반환

## 9. 권장 프로젝트 구조

```text
upbit-xrp-ai/
├─ controller/
│  └─ XrpController.kt
├─ service/
│  ├─ UpbitService.kt
│  ├─ IndicatorService.kt
│  └─ XrpAiService.kt
├─ config/
│  └─ OllamaConfig.kt
├─ dto/
│  ├─ CandleDto.kt
│  └─ PredictionDto.kt
└─ UpbitAiApplication.kt
```

## 10. 실행 흐름

```text
GET http://localhost:8080/api/xrp/predict

        ↓
UpbitService
        ↓
IndicatorService
MA5 / MA20 / RSI / 거래량
        ↓
LangChain4j
        ↓
Ollama
http://localhost:11434
        ↓
Gemma3
        ↓
상승 / 하락 / 횡보 분석
        ↓
REST API 결과
```

## 11. 학습 순서

1단계: Ollama + Gemma3 연결 → 2단계: LangChain4j chat() 호출 → 3단계: REST Controller 작성 → 4단계: 업비트 실제 데이터 연결 → 5단계: 기술지표 계산 → 6단계: DTO로 JSON 결과 정리 → 7단계: 필요할 때 MariaDB/JPA를 추가합니다.

주의: LLM인 Gemma3의 답변을 미래 가격의 확정적 예측으로 사용하면 안 됩니다. 기술지표와 시장 데이터를 해석·요약하는 보조 분석 도구로 사용하는 것이 적절합니다.

---

부록 · LangChain4j · Spring AI

# Spring Boot Java + Kotlin XRP 멀티 타임프레임 분석

15분·60분·240분·일봉·주봉·월봉·연봉 → 지표 → 신호점수 → Gemma3

## 1. 전체 구조

```text
Upbit KRW-XRP
  ├─ 15분봉  → 초단기
  ├─ 60분봉  → 단기
  ├─ 240분봉 → 중단기
  ├─ 일봉     → 중기
  ├─ 주봉     → 장기
  ├─ 월봉     → 장기
  └─ 연봉     → 초장기
          ↓
각 프레임 IndicatorService
          ↓
각 프레임 SignalService
          ↓
MultiTimeframeService
          ↓
가중 종합 상승/하락 점수
          ↓
LangChain4j → Ollama → Gemma3
          ↓
초단기/단기/중기/장기 종합 설명
```

## 2. 시간봉과 역할 비교

| 프레임 | 용도 | 예제 가중치 |
|---|---|---|
| 15분 | 초단기 흐름 | 5% |
| 60분 | 단기 흐름 | 10% |
| 240분 | 중단기 흐름 | 15% |
| 일봉 | 중기 방향 | 25% |
| 주봉 | 장기 추세 | 20% |
| 월봉 | 장기 추세 | 15% |
| 연봉 | 초장기 위치 | 10% |

가중치는 학습용 예제이며 실제 적용 전 백테스트로 조정하는 것이 좋습니다.

## 3. TimeframeAnalysisDto

### Java

```text
public record TimeframeAnalysisDto(
    String timeframe,
    IndicatorDto indicators,
    SignalDto signal
) {}
```

### Kotlin

```kotlin
data class TimeframeAnalysisDto(
    val timeframe: String,
    val indicators: IndicatorDto,
    val signal: SignalDto
)
```

## 4. UpbitService - Java

```java
@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.upbit.com")
            .build();
    }

    public List<CandleDto> getMinuteCandles(int minute) {
        return getCandles("/v1/candles/minutes/" + minute);
    }

    public List<CandleDto> getDayCandles() {
        return getCandles("/v1/candles/days");
    }

    public List<CandleDto> getWeekCandles() {
        return getCandles("/v1/candles/weeks");
    }

    public List<CandleDto> getMonthCandles() {
        return getCandles("/v1/candles/months");
    }

    public List<CandleDto> getYearCandles() {
        return getCandles("/v1/candles/years");
    }

    private List<CandleDto> getCandles(String path) {

        CandleDto[] result = restClient.get()
            .uri(b -> b.path(path)
                .queryParam("market", "KRW-XRP")
                .queryParam("count", 200)
                .build())
            .retrieve()
            .body(CandleDto[].class);

        if (result == null || result.length == 0) {
            throw new IllegalStateException("캔들 데이터가 없습니다: " + path);
        }

        List<CandleDto> list =
            new ArrayList<>(Arrays.asList(result));

        Collections.reverse(list);
        return list;
    }
}
```

## 5. UpbitService - Kotlin

```kotlin
@Service
class UpbitService(
    builder: RestClient.Builder
) {
    private val restClient =
        builder.baseUrl("https://api.upbit.com").build()

    fun getMinuteCandles(minute: Int) =
        getCandles("/v1/candles/minutes/$minute")

    fun getDayCandles() =
        getCandles("/v1/candles/days")

    fun getWeekCandles() =
        getCandles("/v1/candles/weeks")

    fun getMonthCandles() =
        getCandles("/v1/candles/months")

    fun getYearCandles() =
        getCandles("/v1/candles/years")

    private fun getCandles(path: String): List<CandleDto> {

        val result = restClient.get()
            .uri { b ->
                b.path(path)
                    .queryParam("market", "KRW-XRP")
                    .queryParam("count", 200)
                    .build()
            }
            .retrieve()
            .body(Array<CandleDto>::class.java)
            ?: error("캔들 데이터가 없습니다: $path")

        return result.toList().reversed()
    }
}
```

## 6. MultiTimeframeService - Java

```java
@Service
public class MultiTimeframeService {

    private final UpbitService upbitService;
    private final IndicatorService indicatorService;
    private final SignalService signalService;

    public MultiTimeframeService(
            UpbitService upbitService,
            IndicatorService indicatorService,
            SignalService signalService) {
        this.upbitService = upbitService;
        this.indicatorService = indicatorService;
        this.signalService = signalService;
    }

    private TimeframeAnalysisDto analyze(
            String timeframe,
            List<CandleDto> candles) {

        IndicatorDto indicators =
            indicatorService.calculate(candles);

        SignalDto signal =
            signalService.calculate(indicators);

        return new TimeframeAnalysisDto(
            timeframe, indicators, signal);
    }

    public List<TimeframeAnalysisDto> analyzeAll() {
        return List.of(
            analyze("15m",  upbitService.getMinuteCandles(15)),
            analyze("60m",  upbitService.getMinuteCandles(60)),
            analyze("240m", upbitService.getMinuteCandles(240)),
            analyze("1D",   upbitService.getDayCandles()),
            analyze("1W",   upbitService.getWeekCandles()),
            analyze("1M",   upbitService.getMonthCandles()),
            analyze("1Y",   upbitService.getYearCandles())
        );
    }
}
```

## 7. MultiTimeframeService - Kotlin

```kotlin
@Service
class MultiTimeframeService(
    private val upbitService: UpbitService,
    private val indicatorService: IndicatorService,
    private val signalService: SignalService
) {
    private fun analyze(
        timeframe: String,
        candles: List<CandleDto>
    ): TimeframeAnalysisDto {

        val indicators =
            indicatorService.calculate(candles)

        val signal =
            signalService.calculate(indicators)

        return TimeframeAnalysisDto(
            timeframe = timeframe,
            indicators = indicators,
            signal = signal
        )
    }

    fun analyzeAll(): List<TimeframeAnalysisDto> =
        listOf(
            analyze("15m",  upbitService.getMinuteCandles(15)),
            analyze("60m",  upbitService.getMinuteCandles(60)),
            analyze("240m", upbitService.getMinuteCandles(240)),
            analyze("1D",   upbitService.getDayCandles()),
            analyze("1W",   upbitService.getWeekCandles()),
            analyze("1M",   upbitService.getMonthCandles()),
            analyze("1Y",   upbitService.getYearCandles())
        )
}
```

## 8. 종합 결과 DTO

### Java

```text
public record MultiTimeframeSignalDto(
    double bullishScore,
    double bearishScore,
    String signal,
    String strength
) {}
```

### Kotlin

```kotlin
data class MultiTimeframeSignalDto(
    val bullishScore: Double,
    val bearishScore: Double,
    val signal: String,
    val strength: String
)
```

## 9. 가중 종합 점수 - Java

```java
public MultiTimeframeSignalDto calculateTotal(
        List<TimeframeAnalysisDto> list) {

    Map<String, Double> weights = Map.of(
        "15m",  0.05,
        "60m",  0.10,
        "240m", 0.15,
        "1D",   0.25,
        "1W",   0.20,
        "1M",   0.15,
        "1Y",   0.10
    );

    double bullish = 0;
    double bearish = 0;

    for (TimeframeAnalysisDto tf : list) {

        double weight =
            weights.getOrDefault(tf.timeframe(), 0.0);

        bullish +=
            tf.signal().bullishScore() * weight;

        bearish +=
            tf.signal().bearishScore() * weight;
    }

    String signal;
    if (bullish - bearish >= 15) {
        signal = "BULLISH";
    } else if (bearish - bullish >= 15) {
        signal = "BEARISH";
    } else {
        signal = "NEUTRAL";
    }

    double diff = Math.abs(bullish - bearish);

    String strength;
    if (diff >= 40) {
        strength = "STRONG";
    } else if (diff >= 20) {
        strength = "MEDIUM";
    } else {
        strength = "WEAK";
    }

    return new MultiTimeframeSignalDto(
        bullish, bearish, signal, strength);
}
```

## 10. 가중 종합 점수 - Kotlin

```kotlin
fun calculateTotal(
    list: List<TimeframeAnalysisDto>
): MultiTimeframeSignalDto {

    val weights = mapOf(
        "15m"  to 0.05,
        "60m"  to 0.10,
        "240m" to 0.15,
        "1D"   to 0.25,
        "1W"   to 0.20,
        "1M"   to 0.15,
        "1Y"   to 0.10
    )

    var bullish = 0.0
    var bearish = 0.0

    list.forEach { tf ->
        val weight = weights[tf.timeframe] ?: 0.0
        bullish += tf.signal.bullishScore * weight
        bearish += tf.signal.bearishScore * weight
    }

    val signal = when {
        bullish - bearish >= 15 -> "BULLISH"
        bearish - bullish >= 15 -> "BEARISH"
        else -> "NEUTRAL"
    }

    val diff = kotlin.math.abs(bullish - bearish)

    val strength = when {
        diff >= 40 -> "STRONG"
        diff >= 20 -> "MEDIUM"
        else -> "WEAK"
    }

    return MultiTimeframeSignalDto(
        bullishScore = bullish,
        bearishScore = bearish,
        signal = signal,
        strength = strength
    )
}
```

## 11. Gemma3에는 요약 결과만 전달

```text
예:
15m  상승=80 하락=20 BULLISH
60m  상승=90 하락=10 BULLISH
240m 상승=75 하락=25 BULLISH
1D   상승=70 하락=30 BULLISH
1W   상승=60 하락=40 BULLISH
1M   상승=45 하락=55 NEUTRAL/BEARISH
1Y   상승=55 하락=45 NEUTRAL

종합 상승점수=66
종합 하락점수=31
판정=BULLISH
강도=MEDIUM

→ 1,400개 원시 캔들을 LLM에 보내지 않고 위 요약 결과를 전달합니다.
```

## 12. Java - Gemma3 멀티프레임 설명

```java
public String analyze(
        List<TimeframeAnalysisDto> timeframes,
        MultiTimeframeSignalDto total) {

    StringBuilder data = new StringBuilder();

    for (TimeframeAnalysisDto tf : timeframes) {
        data.append(tf.timeframe())
            .append(" 상승=")
            .append(tf.signal().bullishScore())
            .append(" 하락=")
            .append(tf.signal().bearishScore())
            .append(" 판정=")
            .append(tf.signal().signal())
            .append("\n");
    }

    String prompt = """
        당신은 XRP 멀티 타임프레임 기술적 분석 설명 AI입니다.

        아래 결과는 Spring Boot가 직접 계산한 결과입니다.

        %s

        종합 상승점수: %.2f
        종합 하락점수: %.2f
        종합 판정: %s
        신호 강도: %s

        초단기 전망:
        단기 전망:
        중기 전망:
        장기 전망:
        초장기 전망:
        시간봉 간 일치 여부:
        상승 근거:
        하락 위험:
        종합 판단:

        점수를 확률이라고 표현하지 마세요.
        미래 가격을 확정적으로 예측하지 마세요.
        """.formatted(
            data,
            total.bullishScore(),
            total.bearishScore(),
            total.signal(),
            total.strength()
        );

    return chatModel.chat(prompt);
}
```

## 13. Kotlin - Gemma3 멀티프레임 설명

```kotlin
fun analyze(
    timeframes: List<TimeframeAnalysisDto>,
    total: MultiTimeframeSignalDto
): String {

    val data = timeframes.joinToString("\n") {
        "${it.timeframe} " +
        "상승=${it.signal.bullishScore} " +
        "하락=${it.signal.bearishScore} " +
        "판정=${it.signal.signal}"
    }

    val prompt = """
        당신은 XRP 멀티 타임프레임 기술적 분석 설명 AI입니다.

        아래 결과는 Spring Boot가 직접 계산한 결과입니다.

        $data

        종합 상승점수: ${total.bullishScore}
        종합 하락점수: ${total.bearishScore}
        종합 판정: ${total.signal}
        신호 강도: ${total.strength}

        초단기 전망:
        단기 전망:
        중기 전망:
        장기 전망:
        초장기 전망:
        시간봉 간 일치 여부:
        상승 근거:
        하락 위험:
        종합 판단:

        점수를 확률이라고 표현하지 마세요.
        미래 가격을 확정적으로 예측하지 마세요.
    """.trimIndent()

    return chatModel.chat(prompt)
}
```

## 14. Controller

### Java

```java
@GetMapping("/multi-predict")
public Map<String, Object> multiPredict() {

    List<TimeframeAnalysisDto> timeframes =
        multiTimeframeService.analyzeAll();

    MultiTimeframeSignalDto total =
        multiTimeframeService.calculateTotal(timeframes);

    String aiAnalysis =
        aiService.analyze(timeframes, total);

    return Map.of(
        "timeframes", timeframes,
        "total", total,
        "aiAnalysis", aiAnalysis
    );
}
```

### Kotlin

```kotlin
@GetMapping("/multi-predict")
fun multiPredict(): Map<String, Any> {

    val timeframes =
        multiTimeframeService.analyzeAll()

    val total =
        multiTimeframeService.calculateTotal(timeframes)

    val aiAnalysis =
        aiService.analyze(timeframes, total)

    return mapOf(
        "timeframes" to timeframes,
        "total" to total,
        "aiAnalysis" to aiAnalysis
    )
}
```

## 15. Java ↔ Kotlin 비교표

| 항목 | Java | Kotlin |
|---|---|---|
| DTO | record | data class |
| 목록 | List.of() | listOf() |
| 가중치 | Map.of() | mapOf() |
| 반복 | for | forEach |
| 문자열 조립 | StringBuilder | joinToString() |

| 항목 | Java | Kotlin |
|---|---|---|
| DTO 값 접근 | signal() | signal |
| 분기 | if / else | when |
| Spring 구조 | 동일 | 동일 |
| LangChain4j/Gemma3 | 동일 | 동일 |
| 코드량 | 상대적으로 많음 | 상대적으로 간결 |

## 16. 실전 적용 시 중요한 점

연봉은 XRP의 데이터 개수가 적어 MA20, RSI14, MACD 같은 장기 지표가 충분히 안정적으로 계산되지 않을 수 있습니다. 따라서 연봉은 시가·고가·저가·종가, 전년도 대비 변화율, 장기 고점 대비 위치 같은 보조 정보로 쓰고 기술지표 점수는 15분·60분·240분·일봉·주봉·월봉 중심으로 구성하는 편이 좋습니다.

멀티 타임프레임 점수는 확률이 아닙니다. 가중치는 임의의 학습용 값이므로 실제 적용에서는 MariaDB에 예측시각, 프레임별 지표와 점수, 1시간·4시간·1일 후 실제 가격을 저장하여 백테스트로 조정하는 것이 다음 단계입니다.

부록 · LangChain4j · Spring AI

# Spring Boot Java + Kotlin 실전 예제

XRP 60분봉 200개 → MA5 · MA20 · RSI14 → LangChain4j → Ollama Gemma3

```yaml
Ollama: http://localhost:11434
```

## 1. 전체 흐름

```text
업비트 KRW-XRP 60분봉 200개
        ↓
종가(trade_price) 추출
        ↓
MA5 / MA20 / RSI14 계산
        ↓
LangChain4j
        ↓
Ollama  http://localhost:11434
        ↓
Gemma3
        ↓
상승 / 하락 / 횡보 분석
        ↓
Spring Boot REST API
```

## 2. 프로젝트 구조

```text
upbit-ai/
├─ config/
│  └─ OllamaConfig
├─ controller/
│  └─ XrpController
├─ dto/
│  ├─ CandleDto
│  └─ XrpAnalysisDto
├─ service/
│  ├─ UpbitService
│  ├─ IndicatorService
│  └─ XrpAiService
└─ UpbitAiApplication
```

## 3. 클래스 역할 비교

| 클래스 | 역할 |
| --- | --- |
| CandleDto | 업비트 캔들 JSON을 객체로 받음 |
| UpbitService | KRW-XRP 60분봉 200개 조회 |
| IndicatorService | MA5, MA20, RSI14 계산 |
| XrpAiService | 계산 결과를 Gemma3에 전달 |

| 클래스 | 역할 |
| --- | --- |
| XrpController | /api/xrp/predict REST API 제공 |

## 4. application.properties

```properties
server.port=8080

ollama.base-url=http://localhost:11434
ollama.model=gemma3
```

실제 Ollama 모델명이 gemma3:4b 등이라면 ollama.model 값을 실제 설치 이름으로 변경합니다.

## 5. Java Spring Boot

### CandleDto.java

```java
package com.wiz.upbit.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public record CandleDto(
        String market,
        @JsonProperty("candle_date_time_kst") String candleDateTimeKst,
        @JsonProperty("opening_price") double openingPrice,
        @JsonProperty("high_price") double highPrice,
        @JsonProperty("low_price") double lowPrice,
        @JsonProperty("trade_price") double tradePrice,
        @JsonProperty("candle_acc_trade_volume") double volume
) {}
```

### UpbitService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.CandleDto;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import java.util.*;

@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
                .baseUrl("https://api.upbit.com")
                .build();
    }

    public List<CandleDto> getXrpCandles() {
        CandleDto[] result = restClient.get()
                .uri(uriBuilder -> uriBuilder
                        .path("/v1/candles/minutes/60")
                        .queryParam("market", "KRW-XRP")
                        .queryParam("count", 200)
                        .build())
                .retrieve()
                .body(CandleDto[].class);

        if (result == null || result.length == 0) {
            throw new IllegalStateException("업비트 XRP 캔들 데이터가 없습니다.");
        }

        List<CandleDto> candles =
                new ArrayList<>(Arrays.asList(result));

        Collections.reverse(candles);
        return candles;
    }
}
```

### IndicatorService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.CandleDto;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class IndicatorService {

    public double ma(List<CandleDto> candles, int period) {
        int size = candles.size();

        return candles.subList(size - period, size)
                .stream()
                .mapToDouble(CandleDto::tradePrice)
                .average()
                .orElse(0);
    }

    public double rsi(List<CandleDto> candles, int period) {
        int size = candles.size();
        double gain = 0;
        double loss = 0;

        for (int i = size - period; i < size; i++) {
            double current = candles.get(i).tradePrice();
            double previous = candles.get(i - 1).tradePrice();
            double change = current - previous;

            if (change > 0) gain += change;
            else loss += Math.abs(change);
        }

        double avgGain = gain / period;
        double avgLoss = loss / period;

        if (avgLoss == 0) return 100;

        double rs = avgGain / avgLoss;
        return 100 - (100 / (1 + rs));
    }
}
```

### XrpAnalysisDto.java

```java
package com.wiz.upbit.dto;

public record XrpAnalysisDto(
        double price,
        double ma5,
        double ma20,
        double rsi,
        String aiAnalysis
) {}
```

### OllamaConfig.java

```java
package com.wiz.upbit.config;

import dev.langchain4j.model.chat.ChatModel;
import dev.langchain4j.model.ollama.OllamaChatModel;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;

@Configuration
public class OllamaConfig {

    @Bean
    public ChatModel chatModel(
            @Value("${ollama.base-url}") String baseUrl,
            @Value("${ollama.model}") String modelName) {

        return OllamaChatModel.builder()
                .baseUrl(baseUrl)
                .modelName(modelName)
                .temperature(0.2)
                .build();
    }
}
```

### XrpAiService.java

```java
package com.wiz.upbit.service;

import dev.langchain4j.model.chat.ChatModel;
import org.springframework.stereotype.Service;

@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(double price, double ma5,
                          double ma20, double rsi) {

        String prompt = """
            당신은 암호화폐 기술적 분석 보조 AI입니다.
            업비트 XRP 60분봉 데이터입니다.

            현재가격: %.2f
            MA5: %.2f
            MA20: %.2f
            RSI14: %.2f

            다음 형식으로 간단하게 분석하세요.
            추세:
            예상 방향: 상승 / 하락 / 횡보
            분석 이유:
            주의사항:

            확정적인 투자 조언은 하지 마세요.
            """.formatted(price, ma5, ma20, rsi);

        return chatModel.chat(prompt);
    }
}
```

### XrpController.java

```java
package com.wiz.upbit.controller;

import com.wiz.upbit.dto.*;
import com.wiz.upbit.service.*;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final UpbitService upbitService;
    private final IndicatorService indicatorService;
    private final XrpAiService aiService;

    public XrpController(UpbitService upbitService,
                         IndicatorService indicatorService,
                         XrpAiService aiService) {
        this.upbitService = upbitService;
        this.indicatorService = indicatorService;
        this.aiService = aiService;
    }

    @GetMapping("/predict")
    public XrpAnalysisDto predict() {
        List<CandleDto> candles = upbitService.getXrpCandles();

        double price = candles.get(candles.size() - 1).tradePrice();
        double ma5 = indicatorService.ma(candles, 5);
        double ma20 = indicatorService.ma(candles, 20);
        double rsi = indicatorService.rsi(candles, 14);

        String ai = aiService.analyze(price, ma5, ma20, rsi);

        return new XrpAnalysisDto(price, ma5, ma20, rsi, ai);
    }
}
```

## 6. Kotlin Spring Boot

### CandleDto.kt

```kotlin
package com.wiz.upbit.dto

import com.fasterxml.jackson.annotation.JsonProperty

data class CandleDto(
    val market: String,
    @JsonProperty("candle_date_time_kst")
    val candleDateTimeKst: String,
    @JsonProperty("opening_price")
    val openingPrice: Double,
    @JsonProperty("high_price")
    val highPrice: Double,
    @JsonProperty("low_price")
    val lowPrice: Double,
    @JsonProperty("trade_price")
    val tradePrice: Double,
    @JsonProperty("candle_acc_trade_volume")
    val volume: Double
)
```

### UpbitService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.CandleDto
import org.springframework.stereotype.Service
import org.springframework.web.client.RestClient

@Service
class UpbitService(builder: RestClient.Builder) {

    private val restClient = builder
        .baseUrl("https://api.upbit.com")
        .build()

    fun getXrpCandles(): List<CandleDto> {
        val result = restClient.get()
            .uri { b ->
                b.path("/v1/candles/minutes/60")
                    .queryParam("market", "KRW-XRP")
                    .queryParam("count", 200)
                    .build()
            }
            .retrieve()
            .body(Array<CandleDto>::class.java)
            ?: error("업비트 XRP 캔들 데이터가 없습니다.")

        return result.toList().reversed()
    }
}
```

### IndicatorService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.CandleDto
import org.springframework.stereotype.Service
import kotlin.math.abs

@Service
class IndicatorService {

    fun ma(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.tradePrice }
            .average()

    fun rsi(candles: List<CandleDto>, period: Int): Double {
        val recent = candles.takeLast(period + 1)
        var gain = 0.0
        var loss = 0.0

        for (i in 1 until recent.size) {
            val change =
                recent[i].tradePrice - recent[i - 1].tradePrice

            if (change > 0) gain += change
            else loss += abs(change)
        }

        val avgGain = gain / period
        val avgLoss = loss / period

        if (avgLoss == 0.0) return 100.0

        val rs = avgGain / avgLoss
        return 100 - (100 / (1 + rs))
    }
}
```

### XrpAnalysisDto.kt

```kotlin
package com.wiz.upbit.dto

data class XrpAnalysisDto(
    val price: Double,
    val ma5: Double,
    val ma20: Double,
    val rsi: Double,
    val aiAnalysis: String
)
```

### OllamaConfig.kt

```kotlin
package com.wiz.upbit.config

import dev.langchain4j.model.chat.ChatModel
import dev.langchain4j.model.ollama.OllamaChatModel
import org.springframework.beans.factory.annotation.Value
import org.springframework.context.annotation.*

@Configuration
class OllamaConfig {

    @Bean
    fun chatModel(
        @Value("\${ollama.base-url}") baseUrl: String,
        @Value("\${ollama.model}") modelName: String
    ): ChatModel =
        OllamaChatModel.builder()
            .baseUrl(baseUrl)
            .modelName(modelName)
            .temperature(0.2)
            .build()
}
```

### XrpAiService.kt

```kotlin
package com.wiz.upbit.service

import dev.langchain4j.model.chat.ChatModel
import org.springframework.stereotype.Service

@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(
        price: Double,
        ma5: Double,
        ma20: Double,
        rsi: Double
    ): String {

        val prompt = """
            당신은 암호화폐 기술적 분석 보조 AI입니다.
            업비트 XRP 60분봉 데이터입니다.

            현재가격: $price
            MA5: $ma5
            MA20: $ma20
            RSI14: $rsi

            다음 형식으로 분석하세요.
            추세:
            예상 방향: 상승 / 하락 / 횡보
            분석 이유:
            주의사항:

            확정적인 투자 조언은 하지 마세요.
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

### XrpController.kt

```kotlin
package com.wiz.upbit.controller

import com.wiz.upbit.dto.XrpAnalysisDto
import com.wiz.upbit.service.*
import org.springframework.web.bind.annotation.*

@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val upbitService: UpbitService,
    private val indicatorService: IndicatorService,
    private val aiService: XrpAiService
) {
    @GetMapping("/predict")
    fun predict(): XrpAnalysisDto {
        val candles = upbitService.getXrpCandles()

        val price = candles.last().tradePrice
        val ma5 = indicatorService.ma(candles, 5)
        val ma20 = indicatorService.ma(candles, 20)
        val rsi = indicatorService.rsi(candles, 14)

        val ai = aiService.analyze(price, ma5, ma20, rsi)

        return XrpAnalysisDto(
            price = price,
            ma5 = ma5,
            ma20 = ma20,
            rsi = rsi,
            aiAnalysis = ai
        )
    }
}
```

## 7. Java ↔ Kotlin 핵심 비교표

| 작업 | Java | Kotlin |
|---|---|---|
| DTO | record | data class |
| 마지막 캔들 | get(size - 1) | last() |
| 최근 N개 | subList() | takeLast() |
| 역순 | Collections.reverse() | reversed() |
| 평균 | Stream average() | average() |
| LangChain4j | 동일 | 동일 |
| Ollama/Gemma3 | 동일 | 동일 |

## 8. 실행 결과

Spring Boot 실행 후 다음 주소를 호출합니다.

```text
http://localhost:8080/api/xrp/predict
```

```text
예상 형태:
{
  "price": 4215.0,
  "ma5": 4198.4,
  "ma20": 4135.7,
  "rsi": 63.24,
  "aiAnalysis": "추세: 단기 상승 추세 ..."
}
```

## 9. MA와 RSI를 아주 쉽게 이해하기

MA5 = 가장 최근 5개 캔들의 종가 평균입니다. MA20 = 가장 최근 20개 캔들의 종가 평균입니다.

RSI14는 최근 가격 상승폭과 하락폭을 비교하여 0~100 범위로 나타냅니다. 입문적으로 30 이하는 과매도, 70 이상은 과매수 구간으로 해석하지만 단독 매매 신호로 사용하면 안 됩니다.

## 10. 중요한 설계 원칙

200개 캔들 원문 전체를 Gemma3에게 보내기보다 Spring Boot에서 MA·RSI 같은 수치를 먼저 계산하고, 계산 결과를 LLM에 전달하여 해석시키는 구조가 효율적입니다.

이 문서의 RSI는 학습용 단순 평균 방식입니다. 실전 차트와 맞추려면 다음 단계에서 Wilder RSI를 구현하고 EMA, MACD, 볼린저밴드, 거래량 지표 등을 추가할 수 있습니다.

Gemma3의 출력은 기술지표 해석 보조용이며 미래 가격이나 투자 수익을 보장하지 않습니다.

---

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · LangChain4j · Spring AI

# Spring Boot Java + Kotlin 기술지표·Gemma3 실전 예제

Wilder RSI · MA · EMA · MACD · 볼린저밴드 · 거래량 → LangChain4j → Ollama Gemma3

```yaml
Ollama: http://localhost:11434
```

## 1. 권장 전체 구조

```text
업비트 KRW-XRP 60분봉 200개
        ↓
Spring Boot / IndicatorService
 ├─ MA5 / MA20
 ├─ EMA12 / EMA26
 ├─ Wilder RSI14
 ├─ MACD(12,26,9)
 ├─ Bollinger Band(20, 2σ)
 └─ Volume MA20
        ↓
IndicatorDto (계산된 숫자만 정리)
        ↓
LangChain4j
        ↓
Ollama → Gemma3
        ↓
상승 / 하락 / 횡보 + 지표별 해석
```

핵심 원칙: 200개 캔들을 LLM에 그대로 보내지 않고 Java/Kotlin이 수치 계산을 담당하며, Gemma3는 계산된 지표를 해석하는 역할만 담당합니다.

## 2. Java와 Kotlin 구성 비교

| 구성 | Java | Kotlin |
|---|---|---|
| 데이터 객체 | record | data class |
| 최근 N개 | subList() | takeLast() |
| EMA 반복 | for 루프 | forEach 가능 |
| MACD 결과 | double[] | Triple |
| 계산 담당 | IndicatorService | IndicatorService |
| AI 해석 | XrpAiService | XrpAiService |

## 3. IndicatorDto

### Java - IndicatorDto.java

```text
public record IndicatorDto(
    double price,
    double ma5,
    double ma20,
    double ema12,
    double ema26,
    double rsi14,
    double macd,
    double macdSignal,
    double macdHistogram,
    double bollingerUpper,
    double bollingerMiddle,
    double bollingerLower,
    double volume,
    double volumeMa20
) {}
```

### Kotlin - IndicatorDto.kt

```kotlin
data class IndicatorDto(
    val price: Double,
    val ma5: Double,
    val ma20: Double,
    val ema12: Double,
    val ema26: Double,
    val rsi14: Double,
    val macd: Double,
    val macdSignal: Double,
    val macdHistogram: Double,
    val bollingerUpper: Double,
    val bollingerMiddle: Double,
    val bollingerLower: Double,
    val volume: Double,
    val volumeMa20: Double
)
```

## 4. 지표의 뜻

| 지표 | 기본 설정 | 쉬운 의미 |
|---|---|---|
| MA | 5, 20 | 최근 종가의 단순 평균 |
| EMA | 12, 26 | 최근 가격에 더 큰 가중치를 둔 평균 |
| Wilder RSI | 14 | 상승폭과 하락폭의 상대 강도 |
| MACD | 12,26,9 | EMA12-EMA26 및 Signal과의 차이 |
| 볼린저밴드 | 20, 2σ | MA20 기준 변동성 범위 |
| 거래량 MA | 20 | 현재 거래량과 최근 평균 거래량 비교 |

## 5. Java - IndicatorService 핵심 함수

```java
@Service
public class IndicatorService {

    public double ma(List<CandleDto> candles, int period) {
        return candles.subList(candles.size() - period, candles.size())
                .stream()
                .mapToDouble(CandleDto::tradePrice)
                .average()
                .orElse(0);
    }

    public double ema(List<CandleDto> candles, int period) {
        double k = 2.0 / (period + 1);
        double ema = candles.get(0).tradePrice();

        for (int i = 1; i < candles.size(); i++) {
            double price = candles.get(i).tradePrice();
            ema = price * k + ema * (1 - k);
        }
        return ema;
    }

    public double wilderRsi(List<CandleDto> candles, int period) {
        if (candles.size() <= period) return 0;

        double gain = 0, loss = 0;

        for (int i = 1; i <= period; i++) {
            double change = candles.get(i).tradePrice()
                    - candles.get(i - 1).tradePrice();
            if (change > 0) gain += change;
            else loss += Math.abs(change);
        }

        double avgGain = gain / period;
        double avgLoss = loss / period;

        for (int i = period + 1; i < candles.size(); i++) {
            double change = candles.get(i).tradePrice()
                    - candles.get(i - 1).tradePrice();

            double currentGain = Math.max(change, 0);
            double currentLoss = Math.max(-change, 0);

            avgGain = (avgGain * (period - 1) + currentGain) / period;
            avgLoss = (avgLoss * (period - 1) + currentLoss) / period;
        }

        if (avgLoss == 0) return 100;
        double rs = avgGain / avgLoss;
        return 100 - (100 / (1 + rs));
    }

    public List<Double> emaSeries(List<Double> values, int period) {
        List<Double> result = new ArrayList<>();
```

```
        double k = 2.0 / (period + 1);
        double ema = values.get(0);
        result.add(ema);

        for (int i = 1; i < values.size(); i++) {
            ema = values.get(i) * k + ema * (1 - k);
            result.add(ema);
        }
        return result;
    }

    public double[] macd(List<CandleDto> candles) {
        List<Double> prices = candles.stream()
                .map(CandleDto::tradePrice).toList();

        List<Double> ema12 = emaSeries(prices, 12);
        List<Double> ema26 = emaSeries(prices, 26);
        List<Double> macdSeries = new ArrayList<>();

        for (int i = 0; i < prices.size(); i++) {
            macdSeries.add(ema12.get(i) - ema26.get(i));
        }

        List<Double> signal = emaSeries(macdSeries, 9);
        int last = macdSeries.size() - 1;

        double macd = macdSeries.get(last);
        double signalValue = signal.get(last);

        return new double[]{macd, signalValue, macd - signalValue};
    }

    public double[] bollinger(List<CandleDto> candles, int period) {
        List<CandleDto> recent =
                candles.subList(candles.size() - period, candles.size());

        double middle = recent.stream()
                .mapToDouble(CandleDto::tradePrice)
                .average().orElse(0);

        double variance = recent.stream()
                .mapToDouble(c -> Math.pow(c.tradePrice() - middle, 2))
                .average().orElse(0);

        double std = Math.sqrt(variance);

        return new double[]{
                middle + std * 2,
                middle,
                middle - std * 2
        };
    }

    public double volumeMa(List<CandleDto> candles, int period) {
        return candles.subList(candles.size() - period, candles.size())
                .stream()
                .mapToDouble(CandleDto::volume)
                .average()
```

```
                .orElse(0);
    }
}
```

## 6. Kotlin - IndicatorService 핵심 함수

```kotlin
@Service
class IndicatorService {

    fun ma(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.tradePrice }
            .average()

    fun ema(candles: List<CandleDto>, period: Int): Double {
        val k = 2.0 / (period + 1)
        var ema = candles.first().tradePrice

        candles.drop(1).forEach { candle ->
            ema = candle.tradePrice * k + ema * (1 - k)
        }
        return ema
    }

    fun wilderRsi(candles: List<CandleDto>, period: Int): Double {
        if (candles.size <= period) return 0.0

        var gain = 0.0
        var loss = 0.0

        for (i in 1..period) {
            val change = candles[i].tradePrice - candles[i - 1].tradePrice
            if (change > 0) gain += change
            else loss += kotlin.math.abs(change)
        }

        var avgGain = gain / period
        var avgLoss = loss / period

        for (i in period + 1 until candles.size) {
            val change = candles[i].tradePrice - candles[i - 1].tradePrice
            val currentGain = maxOf(change, 0.0)
            val currentLoss = maxOf(-change, 0.0)

            avgGain = (avgGain * (period - 1) + currentGain) / period
            avgLoss = (avgLoss * (period - 1) + currentLoss) / period
        }

        if (avgLoss == 0.0) return 100.0

        val rs = avgGain / avgLoss
        return 100 - (100 / (1 + rs))
    }

    fun emaSeries(values: List<Double>, period: Int): List<Double> {
        val result = mutableListOf<Double>()
        val k = 2.0 / (period + 1)
        var ema = values.first()
        result += ema

        values.drop(1).forEach { value ->
```

```
            ema = value * k + ema * (1 - k)
            result += ema
        }
        return result
    }

    fun macd(candles: List<CandleDto>): Triple<Double, Double, Double> {
        val prices = candles.map { it.tradePrice }
        val ema12 = emaSeries(prices, 12)
        val ema26 = emaSeries(prices, 26)

        val macdSeries = prices.indices.map { i ->
            ema12[i] - ema26[i]
        }

        val signal = emaSeries(macdSeries, 9)
        val macd = macdSeries.last()
        val signalValue = signal.last()

        return Triple(macd, signalValue, macd - signalValue)
    }

    fun bollinger(
        candles: List<CandleDto>,
        period: Int
    ): Triple<Double, Double, Double> {

        val recent = candles.takeLast(period)
        val middle = recent.map { it.tradePrice }.average()

        val variance = recent.map {
            val diff = it.tradePrice - middle
            diff * diff
        }.average()

        val std = kotlin.math.sqrt(variance)

        return Triple(
            middle + std * 2,
            middle,
            middle - std * 2
        )
    }

    fun volumeMa(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.volume }
            .average()
}
```

## 7. 모든 지표를 한 번에 계산

### Java - calculate()

```java
public IndicatorDto calculate(List<CandleDto> candles) {
    double price = candles.get(candles.size() - 1).tradePrice();

    double ma5 = ma(candles, 5);
    double ma20 = ma(candles, 20);
    double ema12 = ema(candles, 12);
    double ema26 = ema(candles, 26);
    double rsi = wilderRsi(candles, 14);

    double[] macd = macd(candles);
    double[] bb = bollinger(candles, 20);

    double volume = candles.get(candles.size() - 1).volume();
    double volumeMa20 = volumeMa(candles, 20);

    return new IndicatorDto(
        price, ma5, ma20, ema12, ema26, rsi,
        macd[0], macd[1], macd[2],
        bb[0], bb[1], bb[2],
        volume, volumeMa20
    );
}
```

### Kotlin - calculate()

```kotlin
fun calculate(candles: List<CandleDto>): IndicatorDto {
    val price = candles.last().tradePrice
    val ma5 = ma(candles, 5)
    val ma20 = ma(candles, 20)
    val ema12 = ema(candles, 12)
    val ema26 = ema(candles, 26)
    val rsi = wilderRsi(candles, 14)

    val macd = macd(candles)
    val bb = bollinger(candles, 20)

    val volume = candles.last().volume
    val volumeMa20 = volumeMa(candles, 20)

    return IndicatorDto(
        price, ma5, ma20, ema12, ema26, rsi,
        macd.first, macd.second, macd.third,
        bb.first, bb.second, bb.third,
        volume, volumeMa20
    )
}
```

## 8. Gemma3에는 계산 결과만 전달

```text
예:
현재가격 4210
MA5 4198 / MA20 4130
EMA12 4185 / EMA26 4145
Wilder RSI14 63.7
MACD 40.2 / Signal 31.1 / Histogram 9.1
Bollinger Upper 4280 / Middle 4130 / Lower 3980
현재 거래량 3,400,000 / Volume MA20 2,100,000

→ 이 요약 숫자만 LangChain4j를 통해 Gemma3에 전달합니다.
```

## 9. Java - XrpAiService

```java
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(IndicatorDto d) {

        double volumeRatio = d.volumeMa20() == 0
                ? 0
                : d.volume() / d.volumeMa20() * 100;

        String prompt = """
            당신은 XRP 기술적 분석 보조 AI입니다.
            아래 값은 Spring Boot가 계산한 결과입니다.

            현재가격: %.2f
            MA5: %.2f
            MA20: %.2f
            EMA12: %.2f
            EMA26: %.2f
            Wilder RSI14: %.2f
            MACD: %.4f
            Signal: %.4f
            Histogram: %.4f
            Bollinger Upper: %.2f
            Bollinger Middle: %.2f
            Bollinger Lower: %.2f
            현재 거래량: %.2f
            거래량 MA20: %.2f
            거래량 비율: %.1f%%

            추세:
            단기 방향: 상승 / 하락 / 횡보
            MA 분석:
            EMA 분석:
            RSI 분석:
            MACD 분석:
            볼린저밴드 분석:
            거래량 분석:
            종합 분석:
            위험요인:

            미래 가격을 확정적으로 예측하지 마세요.
            """.formatted(
                d.price(), d.ma5(), d.ma20(),
                d.ema12(), d.ema26(), d.rsi14(),
                d.macd(), d.macdSignal(), d.macdHistogram(),
                d.bollingerUpper(), d.bollingerMiddle(), d.bollingerLower(),
                d.volume(), d.volumeMa20(), volumeRatio
            );
```

```
        return chatModel.chat(prompt);
    }
}
```

## 10. Kotlin - XrpAiService

```kotlin
@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(d: IndicatorDto): String {

        val volumeRatio =
            if (d.volumeMa20 == 0.0) 0.0
            else d.volume / d.volumeMa20 * 100

        val prompt = """
            당신은 XRP 기술적 분석 보조 AI입니다.
            아래 값은 Spring Boot가 계산한 결과입니다.

            현재가격: ${d.price}
            MA5: ${d.ma5}
            MA20: ${d.ma20}
            EMA12: ${d.ema12}
            EMA26: ${d.ema26}
            Wilder RSI14: ${d.rsi14}
            MACD: ${d.macd}
            Signal: ${d.macdSignal}
            Histogram: ${d.macdHistogram}
            Bollinger Upper: ${d.bollingerUpper}
            Bollinger Middle: ${d.bollingerMiddle}
            Bollinger Lower: ${d.bollingerLower}
            현재 거래량: ${d.volume}
            거래량 MA20: ${d.volumeMa20}
            거래량 비율: $volumeRatio %

            추세:
            단기 방향: 상승 / 하락 / 횡보
            MA 분석:
            EMA 분석:
            RSI 분석:
            MACD 분석:
            볼린저밴드 분석:
            거래량 분석:
            종합 분석:
            위험요인:

            미래 가격을 확정적으로 예측하지 마세요.
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

## 11. Controller

### Java

```java
@GetMapping("/predict")
public Map<String, Object> predict() {
    List<CandleDto> candles = upbitService.getXrpCandles();
    IndicatorDto indicators = indicatorService.calculate(candles);
    String analysis = aiService.analyze(indicators);

    return Map.of(
        "indicators", indicators,
        "analysis", analysis
    );
}
```

### Kotlin

```kotlin
@GetMapping("/predict")
fun predict(): Map<String, Any> {
    val candles = upbitService.getXrpCandles()
    val indicators = indicatorService.calculate(candles)
    val analysis = aiService.analyze(indicators)

    return mapOf(
        "indicators" to indicators,
        "analysis" to analysis
    )
}
```

## 12. 실행 흐름과 핵심 정리

```text
GET http://localhost:8080/api/xrp/predict
        ↓
UpbitService → XRP 60분봉 200개
        ↓
IndicatorService → 모든 지표 계산
        ↓
IndicatorDto → 계산 결과 요약
        ↓
LangChain4j → Ollama → Gemma3
        ↓
지표별 해석 + 종합 분석
```

권장 설계: Java/Kotlin은 계산 엔진, Gemma3는 분석·설명 엔진으로 역할을 분리합니다. 이 방식은 캔들 원문 전체를 LLM에 보내는 것보다 입력량을 줄이고 계산을 코드로 통제할 수 있다는 장점이 있습니다.

다음 발전 단계는 SignalService를 추가하여 MA·EMA·RSI·MACD·볼린저밴드·거래량 조건별 점수를 계산한 뒤, 그 점수와 지표를 Gemma3가 설명하도록 만드는 것입니다.

주의: 기술지표와 LLM 분석은 미래 수익이나 가격을 보장하지 않으며 투자 판단의 보조 자료로만 사용해야 합니다.

---

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · LangChain4j · Spring AI

# Spring Boot Java + Kotlin 신호점수 실전 예제

IndicatorDto → SignalService → 상승·하락 점수 → LangChain4j → Gemma3

```yaml
Ollama: http://localhost:11434
```

## 1. 전체 구조

```text
업비트 XRP 60분봉 200개
        ↓
UpbitService
        ↓
IndicatorService
        ↓
IndicatorDto
 ├─ MA5 / MA20
 ├─ EMA12 / EMA26
 ├─ Wilder RSI14
 ├─ MACD / Signal / Histogram
 ├─ Bollinger Band
 └─ Volume / VolumeMA20
        ↓
SignalService
        ↓
상승점수 / 하락점수 / 중립점수
BULLISH / BEARISH / NEUTRAL
STRONG / MEDIUM / WEAK
        ↓
LangChain4j
        ↓
Ollama → Gemma3
        ↓
점수와 지표를 자연어로 설명
```

역할 분리: IndicatorService = 지표 계산, SignalService = 규칙·점수 계산, Gemma3 = 계산 결과의 자연어 해석.

## 2. 학습용 점수 규칙

| 조건 | 상승점수 | 하락점수 |
|---|---|---|
| MA5 > MA20 | +20 | - |
| MA5 < MA20 | - | +20 |
| EMA12 > EMA26 | +20 | - |
| EMA12 < EMA26 | - | +20 |
| RSI 50~70 | +15 | - |
| RSI 30~50 | - | +10 |
| RSI ≥ 70 | - | +10 |

| 조건 | 상승점수 | 하락점수 |
|---|---|---|
| RSI ≤ 30 | +10 | - |
| MACD Histogram > 0 | +20 | - |
| MACD Histogram < 0 | - | +20 |
| 가격 > BB Middle | +10 | - |
| 가격 < BB Middle | - | +10 |
| 거래량 ≥ MA20×1.2 | 우세 방향 +15 | 우세 방향 +15 |

주의: 이 점수는 상승·하락 확률이 아니라 학습용 규칙의 충족도입니다.

## 3. SignalDto

### Java - SignalDto.java

```text
public record SignalDto(
    int bullishScore,
    int bearishScore,
    int neutralScore,
    String signal,
    String strength,
    String reasons
) {}
```

### Kotlin - SignalDto.kt

```kotlin
data class SignalDto(
    val bullishScore: Int,
    val bearishScore: Int,
    val neutralScore: Int,
    val signal: String,
    val strength: String,
    val reasons: String
)
```

## 4. Java - SignalService

```java
@Service
public class SignalService {

    public SignalDto calculate(IndicatorDto d) {

        int bullish = 0;
        int bearish = 0;
        List<String> reasons = new ArrayList<>();

        // 1. MA
        if (d.ma5() > d.ma20()) {
            bullish += 20;
            reasons.add("MA5가 MA20보다 높음");
        } else if (d.ma5() < d.ma20()) {
            bearish += 20;
            reasons.add("MA5가 MA20보다 낮음");
        }

        // 2. EMA
        if (d.ema12() > d.ema26()) {
            bullish += 20;
            reasons.add("EMA12가 EMA26보다 높음");
        } else if (d.ema12() < d.ema26()) {
            bearish += 20;
            reasons.add("EMA12가 EMA26보다 낮음");
        }

        // 3. RSI
        if (d.rsi14() >= 50 && d.rsi14() < 70) {
            bullish += 15;
            reasons.add("RSI가 상승 우위 구간");
        } else if (d.rsi14() >= 30 && d.rsi14() < 50) {
            bearish += 10;
            reasons.add("RSI가 약세 구간");
        } else if (d.rsi14() >= 70) {
            bearish += 10;
            reasons.add("RSI 과매수 주의");
        } else if (d.rsi14() <= 30) {
            bullish += 10;
            reasons.add("RSI 과매도 반등 가능성");
        }

        // 4. MACD
        if (d.macdHistogram() > 0) {
            bullish += 20;
            reasons.add("MACD Histogram 양수");
        } else if (d.macdHistogram() < 0) {
            bearish += 20;
            reasons.add("MACD Histogram 음수");
        }

        // 5. Bollinger
        if (d.price() > d.bollingerMiddle()) {
            bullish += 10;
            reasons.add("가격이 볼린저 중앙선 위");
```

```
        } else {
            bearish += 10;
            reasons.add("가격이 볼린저 중앙선 아래");
        }

        // 6. Volume
        double volumeRatio = d.volumeMa20() == 0
                ? 0 : d.volume() / d.volumeMa20();

        if (volumeRatio >= 1.2) {
            if (bullish > bearish) {
                bullish += 15;
                reasons.add("상승 조건에서 거래량 증가");
            } else if (bearish > bullish) {
                bearish += 15;
                reasons.add("하락 조건에서 거래량 증가");
            }
        }

        bullish = Math.min(bullish, 100);
        bearish = Math.min(bearish, 100);

        int neutral = Math.max(
                0, 100 - Math.max(bullish, bearish));

        String signal;
        if (bullish - bearish >= 20) {
            signal = "BULLISH";
        } else if (bearish - bullish >= 20) {
            signal = "BEARISH";
        } else {
            signal = "NEUTRAL";
        }

        int difference = Math.abs(bullish - bearish);

        String strength;
        if (difference >= 50) {
            strength = "STRONG";
        } else if (difference >= 25) {
            strength = "MEDIUM";
        } else {
            strength = "WEAK";
        }

        return new SignalDto(
                bullish, bearish, neutral,
                signal, strength,
                String.join(", ", reasons));
    }
}
```

## 5. Kotlin - SignalService

```kotlin
@Service
class SignalService {

    fun calculate(d: IndicatorDto): SignalDto {

        var bullish = 0
        var bearish = 0
        val reasons = mutableListOf<String>()

        // 1. MA
        if (d.ma5 > d.ma20) {
            bullish += 20
            reasons += "MA5가 MA20보다 높음"
        } else if (d.ma5 < d.ma20) {
            bearish += 20
            reasons += "MA5가 MA20보다 낮음"
        }

        // 2. EMA
        if (d.ema12 > d.ema26) {
            bullish += 20
            reasons += "EMA12가 EMA26보다 높음"
        } else if (d.ema12 < d.ema26) {
            bearish += 20
            reasons += "EMA12가 EMA26보다 낮음"
        }

        // 3. RSI
        when {
            d.rsi14 >= 50.0 && d.rsi14 < 70.0 -> {
                bullish += 15
                reasons += "RSI 상승 우위 구간"
            }
            d.rsi14 >= 30.0 && d.rsi14 < 50.0 -> {
                bearish += 10
                reasons += "RSI 약세 구간"
            }
            d.rsi14 >= 70.0 -> {
                bearish += 10
                reasons += "RSI 과매수 주의"
            }
            d.rsi14 <= 30.0 -> {
                bullish += 10
                reasons += "RSI 과매도 반등 가능성"
            }
        }

        // 4. MACD
        if (d.macdHistogram > 0) {
            bullish += 20
            reasons += "MACD Histogram 양수"
        } else if (d.macdHistogram < 0) {
            bearish += 20
            reasons += "MACD Histogram 음수"
        }
```

```
        // 5. Bollinger
        if (d.price > d.bollingerMiddle) {
            bullish += 10
            reasons += "가격이 볼린저 중앙선 위"
        } else {
            bearish += 10
            reasons += "가격이 볼린저 중앙선 아래"
        }

        // 6. Volume
        val volumeRatio =
            if (d.volumeMa20 == 0.0) 0.0
            else d.volume / d.volumeMa20

        if (volumeRatio >= 1.2) {
            if (bullish > bearish) {
                bullish += 15
                reasons += "상승 조건에서 거래량 증가"
            } else if (bearish > bullish) {
                bearish += 15
                reasons += "하락 조건에서 거래량 증가"
            }
        }

        bullish = bullish.coerceAtMost(100)
        bearish = bearish.coerceAtMost(100)

        val neutral =
            (100 - maxOf(bullish, bearish)).coerceAtLeast(0)

        val signal = when {
            bullish - bearish >= 20 -> "BULLISH"
            bearish - bullish >= 20 -> "BEARISH"
            else -> "NEUTRAL"
        }

        val difference =
            kotlin.math.abs(bullish - bearish)

        val strength = when {
            difference >= 50 -> "STRONG"
            difference >= 25 -> "MEDIUM"
            else -> "WEAK"
        }

        return SignalDto(
            bullishScore = bullish,
            bearishScore = bearish,
            neutralScore = neutral,
            signal = signal,
            strength = strength,
            reasons = reasons.joinToString(", ")
        )
    }
}
```

## 6. 계산 예시

```text
입력 지표 예:
현재 XRP = 4,210
MA5 = 4,190 / MA20 = 4,130
EMA12 = 4,185 / EMA26 = 4,145
RSI14 = 62
MACD Histogram = +12
Bollinger Middle = 4,130
현재 거래량 = 3,000,000
Volume MA20 = 2,000,000

점수:
MA +20
EMA +20
RSI +15
MACD +20
Bollinger +10
거래량 증가 +15
----------------
상승점수 = 100
```

100점은 '상승확률 100%'가 아니라 정의한 규칙을 강하게 충족했다는 뜻입니다.

## 7. Java - Gemma3 분석 Service

```java
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(IndicatorDto i, SignalDto s) {

        String prompt = """
            당신은 XRP 기술적 분석 설명 AI입니다.

            [기술지표]
            현재가격: %.2f
            MA5: %.2f
            MA20: %.2f
            EMA12: %.2f
            EMA26: %.2f
            Wilder RSI14: %.2f
            MACD: %.4f
            Signal: %.4f
            Histogram: %.4f
            Bollinger Upper: %.2f
            Bollinger Middle: %.2f
            Bollinger Lower: %.2f

            [Spring Boot 신호 계산]
            상승점수: %d
            하락점수: %d
            중립점수: %d
            판정: %s
            강도: %s
            계산 근거: %s

            계산 결과를 변경하지 말고 이해하기 쉽게 설명하세요.

            현재 추세:
            지표 해석:
            상승 근거:
            하락 위험:
            거래량 해석:
            종합 의견:
            주의사항:

            점수를 상승 확률로 표현하지 마세요.
            확정적인 투자 조언을 하지 마세요.
            """.formatted(
                i.price(), i.ma5(), i.ma20(),
                i.ema12(), i.ema26(), i.rsi14(),
                i.macd(), i.macdSignal(), i.macdHistogram(),
                i.bollingerUpper(), i.bollingerMiddle(), i.bollingerLower(),
                s.bullishScore(), s.bearishScore(), s.neutralScore(),
                s.signal(), s.strength(), s.reasons()
```

```
            );

        return chatModel.chat(prompt);
    }
}
```

## 8. Kotlin - Gemma3 분석 Service

```kotlin
@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(i: IndicatorDto, s: SignalDto): String {

        val prompt = """
            당신은 XRP 기술적 분석 설명 AI입니다.

            [기술지표]
            현재가격: ${i.price}
            MA5: ${i.ma5}
            MA20: ${i.ma20}
            EMA12: ${i.ema12}
            EMA26: ${i.ema26}
            Wilder RSI14: ${i.rsi14}
            MACD: ${i.macd}
            Signal: ${i.macdSignal}
            Histogram: ${i.macdHistogram}
            Bollinger Upper: ${i.bollingerUpper}
            Bollinger Middle: ${i.bollingerMiddle}
            Bollinger Lower: ${i.bollingerLower}

            [Spring Boot 신호 계산]
            상승점수: ${s.bullishScore}
            하락점수: ${s.bearishScore}
            중립점수: ${s.neutralScore}
            판정: ${s.signal}
            강도: ${s.strength}
            계산 근거: ${s.reasons}

            계산 결과를 변경하지 말고 이해하기 쉽게 설명하세요.

            현재 추세:
            지표 해석:
            상승 근거:
            하락 위험:
            거래량 해석:
            종합 의견:
            주의사항:

            점수를 상승 확률로 표현하지 마세요.
            확정적인 투자 조언을 하지 마세요.
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

## 9. Controller 전체 파이프라인

### Java

```java
@GetMapping("/predict")
public Map<String, Object> predict() {

    List<CandleDto> candles =
            upbitService.getXrpCandles();

    IndicatorDto indicators =
            indicatorService.calculate(candles);

    SignalDto signal =
            signalService.calculate(indicators);

    String analysis =
            aiService.analyze(indicators, signal);

    return Map.of(
            "indicators", indicators,
            "signal", signal,
            "aiAnalysis", analysis
    );
}
```

### Kotlin

```kotlin
@GetMapping("/predict")
fun predict(): Map<String, Any> {

    val candles =
        upbitService.getXrpCandles()

    val indicators =
        indicatorService.calculate(candles)

    val signal =
        signalService.calculate(indicators)

    val analysis =
        aiService.analyze(indicators, signal)

    return mapOf(
        "indicators" to indicators,
        "signal" to signal,
        "aiAnalysis" to analysis
    )
}
```

## 10. 최종 실행 흐름

```text
GET http://localhost:8080/api/xrp/predict
        ↓
UpbitService → XRP 200개
        ↓
IndicatorService → 기술지표
        ↓
IndicatorDto
        ↓
SignalService → 규칙 기반 점수
        ↓
SignalDto
        ↓
LangChain4j
        ↓
Ollama http://localhost:11434
        ↓
Gemma3 → 자연어 설명
```

## 11. Java ↔ Kotlin 핵심 비교

| 항목 | Java | Kotlin |
|---|---|---|
| DTO | record | data class |
| 점수 변수 | int | var |
| 조건 분기 | if / else if | if / when |
| 이유 목록 | List<String> | mutableListOf<String> |
| 상한 제한 | Math.min | coerceAtMost |
| DTO 접근 | d.ma5() | d.ma5 |
| Spring 구조 | 동일 | 동일 |
| LangChain4j/Gemma3 | 동일 | 동일 |

## 12. 다음 발전 단계

다음 단계는 15분봉·60분봉·240분봉을 동시에 계산하는 MultiTimeframeService입니다. 각 시간대의 IndicatorDto와 SignalDto를 만든 뒤 단기·중기·큰 추세를 종합할 수 있습니다.

그 다음 MariaDB에 시점별 지표·점수·실제 이후 가격을 저장하면 백테스트를 통해 점수 규칙의 성능을 검증하고 가중치를 조정할 수 있습니다.

주의: 규칙 점수와 Gemma3 설명은 투자수익 또는 미래가격을 보장하지 않습니다.

---

부록 · LangChain4j · Spring AI

# Spring AI vs LangChain4j

Spring Boot Java + Kotlin + Ollama Gemma3 실전 비교

## 1. 한눈에 보는 구조

```text
Spring AI: Spring Boot → Spring AI ChatModel → Ollama → Gemma3
LangChain4j: Spring Boot → LangChain4j ChatModel/AiService → Ollama → Gemma3

핵심: 둘 다 Spring Boot Java/Kotlin에서 사용할 수 있습니다. Spring AI는 Spring 생태계와의 자연스러운 통합이 강점이고, LangChain4j는 AiService·RAG·Tools·Agent 같은 LLM 애플리케이션 구성이 편리합니다.
```

## 2. Spring AI - 설정

```kotlin
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.ai:spring-ai-starter-model-ollama")
}
```

```properties
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.model=gemma3
spring.ai.ollama.chat.temperature=0.2
```

## 3. Spring AI - Java

```java
@RestController
@RequestMapping("/api/ai")
public class SpringAiController {

    private final ChatModel chatModel;

    public SpringAiController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping("/spring-ai")
    public Map<String, String> chat(@RequestParam String message) {
        String answer = chatModel.call(message);
        return Map.of("answer", answer);
    }
}
```

## 4. Spring AI - Kotlin

```kotlin
@RestController
@RequestMapping("/api/ai")
class SpringAiController(
    private val chatModel: ChatModel
) {
    @GetMapping("/spring-ai")
    fun chat(@RequestParam message: String): Map<String, String> {
        val answer = chatModel.call(message)
        return mapOf("answer" to answer)
    }
}
```

## 5. LangChain4j - 기본 구조

LangChain4j에서는 ChatModel을 직접 호출할 수도 있고,

AiService 인터페이스를 만들어 업무 의미가 드러나는 AI 서비스를 구성할 수도 있습니다.

Spring Boot → XrpAssistant → LangChain4j → Ollama → Gemma3

## 6. LangChain4j - Java ChatModel

```java
@RestController
@RequestMapping("/api/ai")
public class LangChainController {

    private final ChatModel chatModel;

    public LangChainController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping("/langchain4j")
    public Map<String, String> chat(@RequestParam String message) {
        String answer = chatModel.chat(message);
        return Map.of("answer", answer);
    }
}
```

## 7. LangChain4j - Kotlin ChatModel

```kotlin
@RestController
@RequestMapping("/api/ai")
class LangChainController(
    private val chatModel: ChatModel
) {
    @GetMapping("/langchain4j")
    fun chat(@RequestParam message: String): Map<String, String> {
        val answer = chatModel.chat(message)
        return mapOf("answer" to answer)
    }
}
```

## 8. LangChain4j AiService - Java

```java
@AiService
public interface XrpAssistant {

    @SystemMessage("""
        당신은 XRP 기술적 분석 설명 AI입니다.
        제공된 기술지표를 이해하기 쉽게 설명하세요.
        확정적인 투자 조언은 하지 마세요.
        """)
    String analyze(String message);
}
```

```java
@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final XrpAssistant assistant;

    public XrpController(XrpAssistant assistant) {
        this.assistant = assistant;
    }

    @GetMapping("/analysis")
    public String analysis() {
        return assistant.analyze("""
            XRP
            MA5=4200
            MA20=4100
            RSI14=63
            MACD Histogram=15
            분석해주세요.
            """);
    }
}
```

## 9. LangChain4j AiService - Kotlin

```kotlin
@AiService
interface XrpAssistant {

    @SystemMessage(
        """
        당신은 XRP 기술적 분석 설명 AI입니다.
        제공된 기술지표를 이해하기 쉽게 설명하세요.
        확정적인 투자 조언은 하지 마세요.
        """
    )
    fun analyze(message: String): String
}
```

```kotlin
@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val assistant: XrpAssistant
) {
    @GetMapping("/analysis")
    fun analysis(): String =
        assistant.analyze(
            """
            XRP
            MA5=4200
            MA20=4100
            RSI14=63
            MACD Histogram=15
            분석해주세요.
            """.trimIndent()
        )
}
```

## 10. XRP 분석 프로젝트에 적용

```text
UpbitService
    ↓
IndicatorService
    ↓
IndicatorDto
    ↓
SignalService
    ↓
SignalDto
    ↓
[선택 A] Spring AI ChatModel
또는
[선택 B] LangChain4j XrpAssistant
    ↓
Ollama (http://localhost:11434)
    ↓
Gemma3
    ↓
자연어 분석
```

## 11. Spring AI vs LangChain4j 특징 비교표

| 항목 | Spring AI | LangChain4j |
|---|---|---|
| 주요 방향 | Spring 중심 AI 통합 | Java 중심 LLM 애플리케이션 |
| Spring Boot 친화성 | 매우 높음 | 높음 |
| Java | 좋음 | 좋음 |
| Kotlin | 좋음 | 좋음 |
| Ollama/Gemma3 | 지원 | 지원 |
| 기본 모델 추상화 | ChatModel | ChatModel |

| 항목 | Spring AI | LangChain4j |
|---|---|---|
| 간단 호출 | call() | chat() |
| 고수준 AI 서비스 | Spring 방식 구성 | AiService가 편리 |
| Prompt | 지원 | 지원 |
| Streaming | 지원 | 지원 |
| Tool Calling | 지원 | 지원 |
| Embedding | 지원 | 지원 |
| Vector Store | 지원 | 지원 |
| RAG | 지원 | 지원 |
| Chat Memory | 지원 | 강점 |
| Agent/Tools 확장 | 가능 | 강점 |
| 초보자 기본 Chat | 쉬움 | 쉬움 |
| 복잡한 AI 서비스 | 좋음 | 매우 편리 |
| XRP 분석 프로젝트 | 적합 | 매우 적합 |

## 12. Java와 Kotlin 비교

| 항목 | Java | Kotlin |
|---|---|---|
| 생성자 주입 | 필드+생성자 | 주 생성자로 간결 |
| Map 반환 | Map.of() | mapOf() |
| 멀티라인 문자열 | text block | triple-quoted string |
| DTO | record | data class |
| Spring AI 기능 | 동일 | 동일 |
| LangChain4j 기능 | 동일 | 동일 |
| 코드량 | 상대적으로 많음 | 상대적으로 적음 |

## 13. 추천 학습 순서

1) Spring Boot REST API
2) Spring AI ChatModel + Ollama/Gemma3
3) LangChain4j ChatModel
4) LangChain4j AiService
5) RAG
6) Chat Memory
7) Tool Calling
8) Agent
9) 기존 XRP IndicatorService/SignalService/MultiTimeframeService와 연결

현재 XRP 프로젝트는 LangChain4j 구조를 계속 유지해도 좋습니다. 학습 목적으로 동일 기능을 Spring AI와 LangChain4j 두 버전으로 구현하면 두 프레임워크의 차이를 가장 빨리 이해할 수 있습니다.

---